LAPORAN PRAKTIK PEMROGRAMAN VISUAL

Pertemuan 8: DataGridView, Array & Koleksi Data

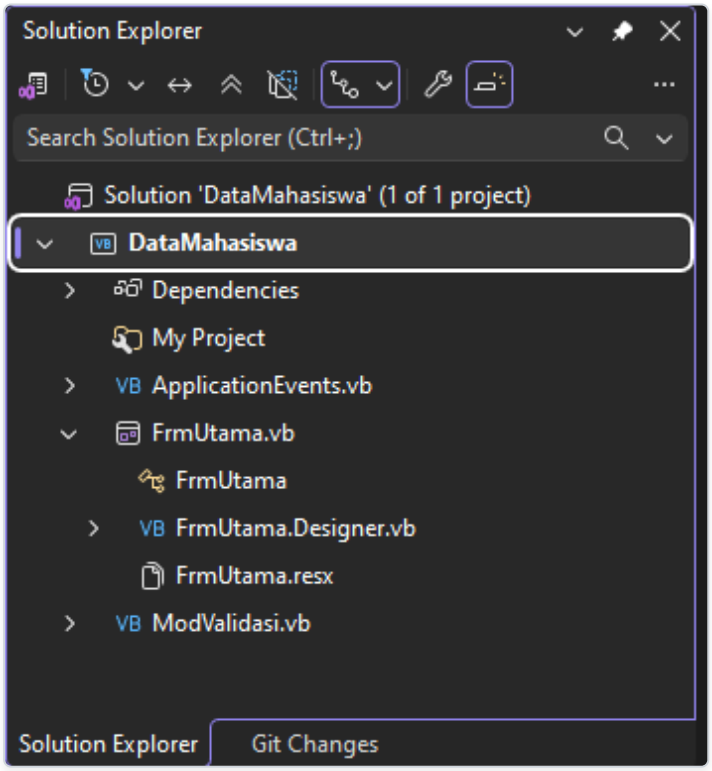
Mata Kuliah: Pemrograman Visual (TIF 3 031) - Semester Ganjil 2026/2027

Materi Praktik	: Praktik 8 - DataGridView, BindingSource, DataTable & Pencarian
Nama Proyek	: DataMahasiswa (Windows Forms App VB.NET .NET 10, Option Strict On)
Status Pengerjaan	: Lengkap (Langkah 0 sampai Praktik 4 beserta seluruh pengujian)

> **Ringkasan Praktik:** Praktik ini membangun aplikasi **DataMahasiswa** dengan satu form utama (`FrmUtama`) dan empat form studi kasus yang membandingkan berbagai cara mengelola data di Windows Forms: melalui Array 2 Dimensi (`FrmDariArray`), List terikat BindingSource (`FrmDariList`), DataTable dengan filter kueri memori (`FrmDataTable`), serta pencarian iteratif pada List (`FrmCariList`).

Langkah 0: Inisialisasi Proyek dan Salin ModValidasi

Proyek dibuat menggunakan template **Windows Forms App** Visual Basic (.NET 10) dengan opsi `Option Strict = On`. Form pembuka dinamai `FrmUtama.vb` dan modul validasi `ModValidasi.vb` disalin dari Pertemuan 7.



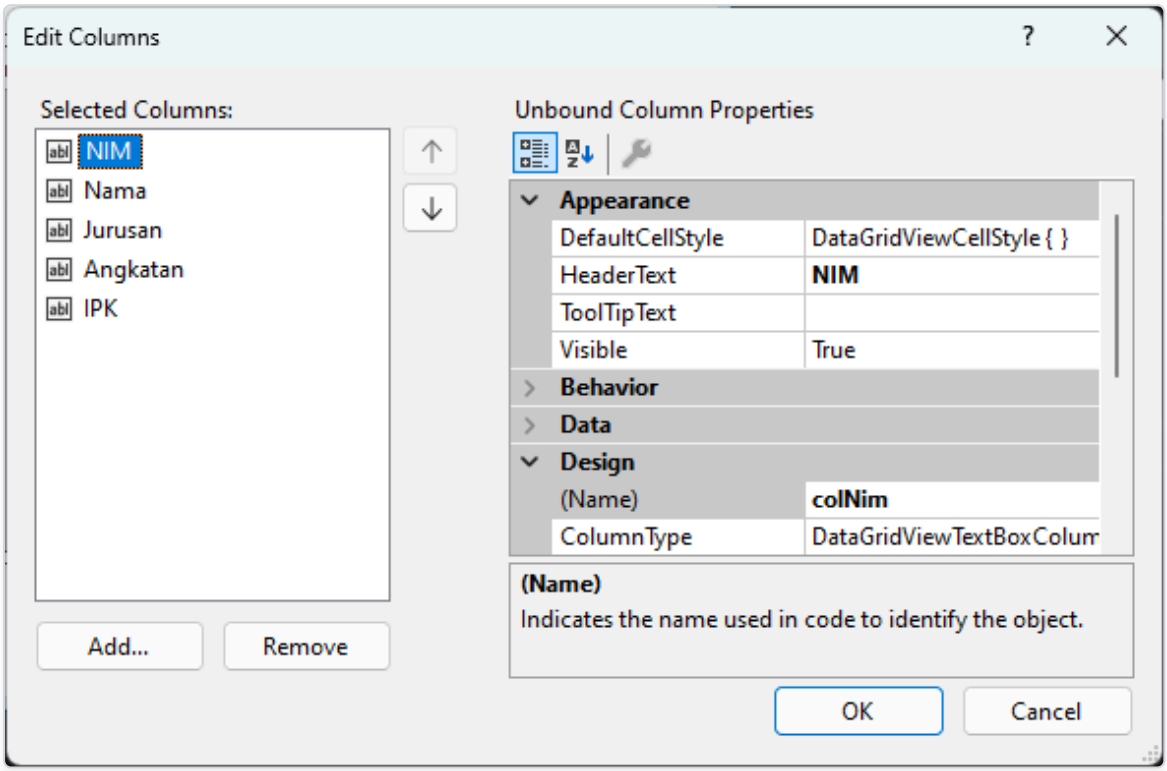
Gambar 1: Solution Explorer proyek DataMahasiswa berisi FrmUtama.vb dan ModValidasi.vb

Praktik 1: DataGridView dari Array 2 Dimensi (FrmDariArray)

Pada praktik ini, DataGridView diisi secara manual menggunakan perulangan bertingkat dari array dua dimensi `dataMahasiswa(,)`. Dimensi pertama mewakili baris dan dimensi kedua mewakili kolom.

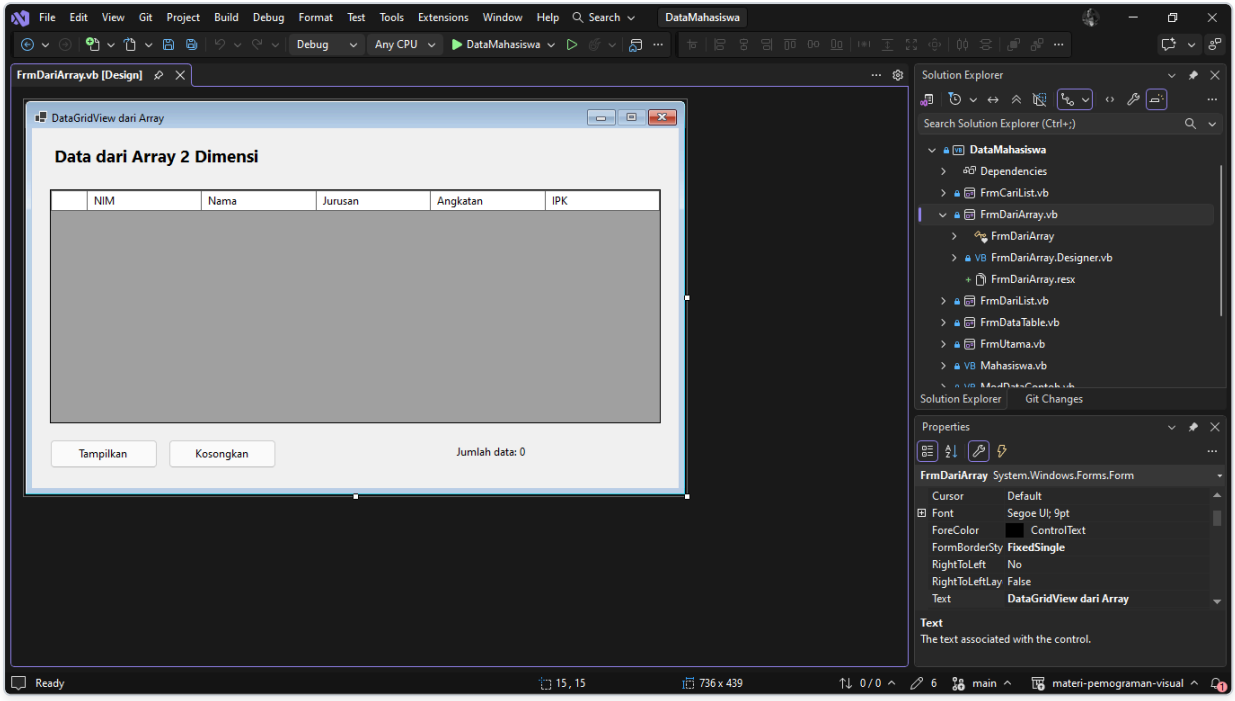
1.1 Perancangan Kolom DataGridView

Lima kolom didefinisikan lewat dialog *Edit Columns*: `colNim` (NIM), `colNama` (Nama), `colJurusan` (Jurusan), `colAngkatan` (Angkatan), dan `colIpk` (IPK).



Gambar 2: Dialog Edit Columns dgvMahasiswa dengan lima kolom terkonfigurasi

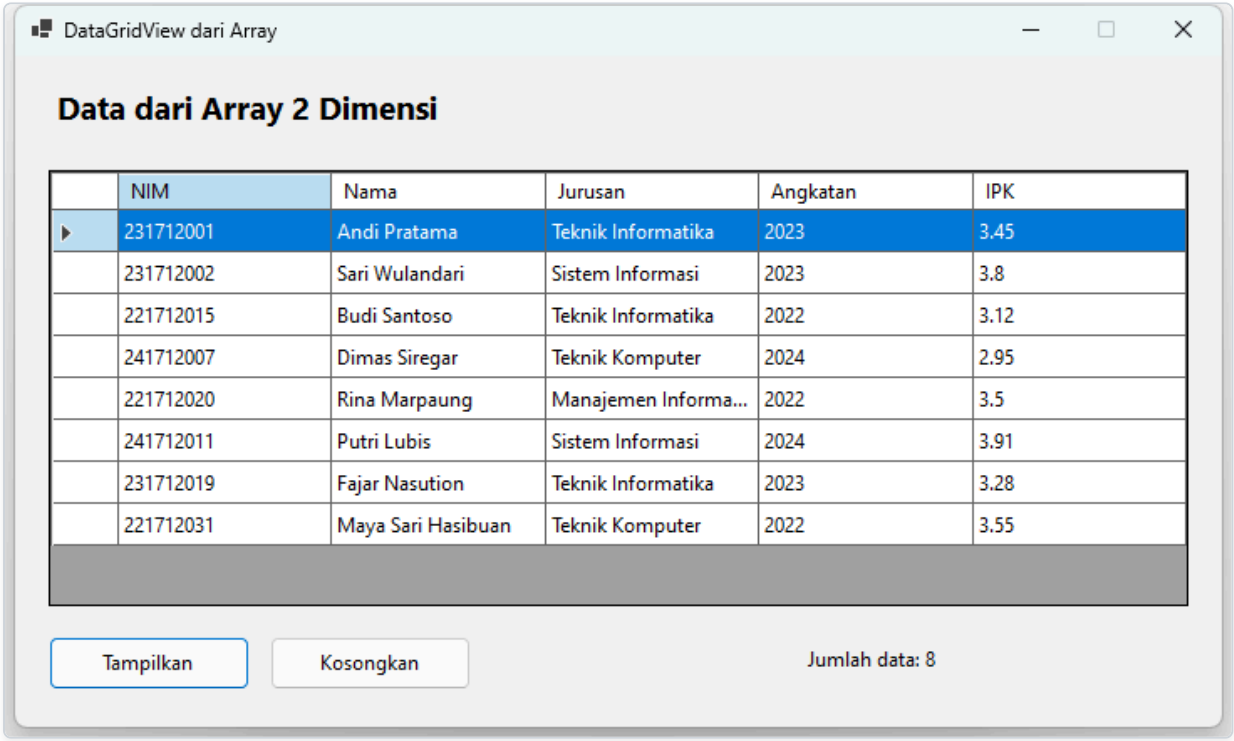
1.2 Desain Form FrmDariArray



Gambar 3: Tampilan form FrmDariArray di Visual Studio Form Designer

1.3 Pengujian Form FrmDariArray

Saat tombol **Tampilkan** diklik, seluruh 8 data dari array berhasil masuk ke grid dan label jumlah menunjukkan `Jumlah data: 8`. Baris terpilih secara penuh (FullRowSelect) dan grid berstatus ReadOnly.



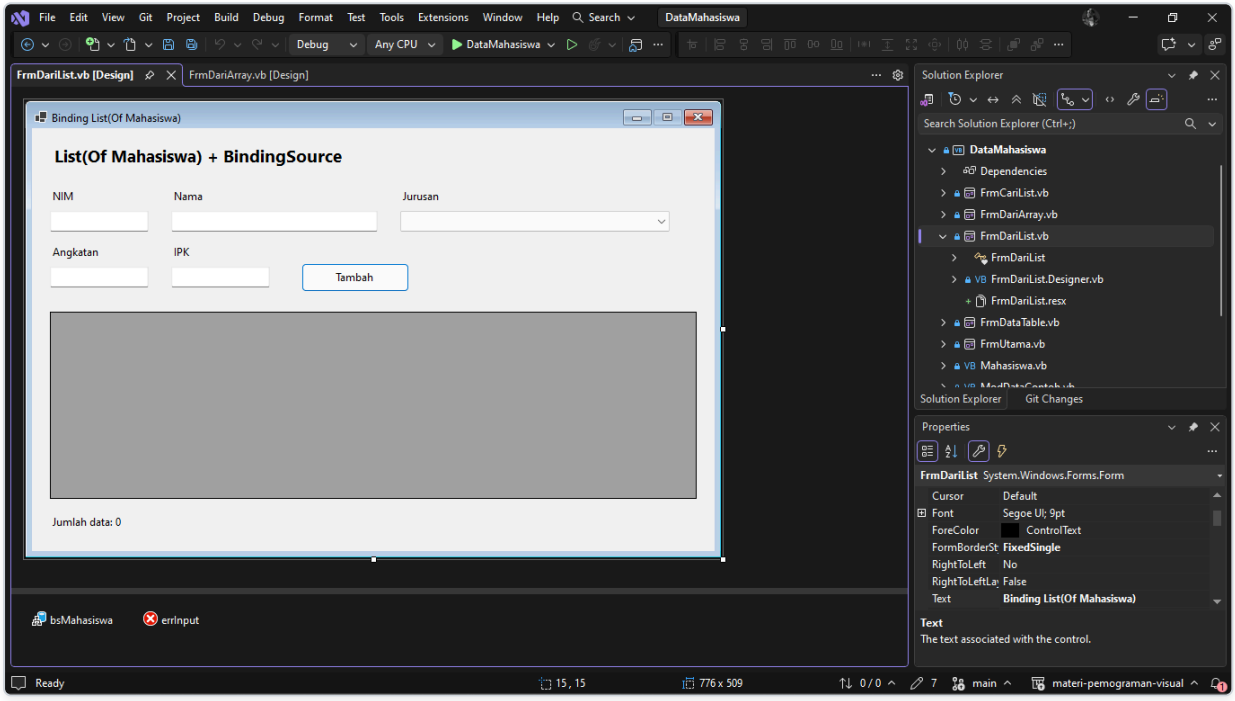
Gambar 4: FrmDariArray setelah tombol Tampilkan diklik, 8 baris data tampil

Praktik 2: List(Of Mahasiswa) dan BindingSource (FrmDariList)

Praktik ini menerapkan konsep Object-Oriented Programming (OOP) dengan membuat class `Mahasiswa` yang memiliki properti Nim, Nama, Jurusan, Angkatan, dan Ipk. Data ditampung dalam `List(Of Mahasiswa)` dan diikat ke DataGridView melalui perantara **BindingSource**.

2.1 Desain Form FrmDariList

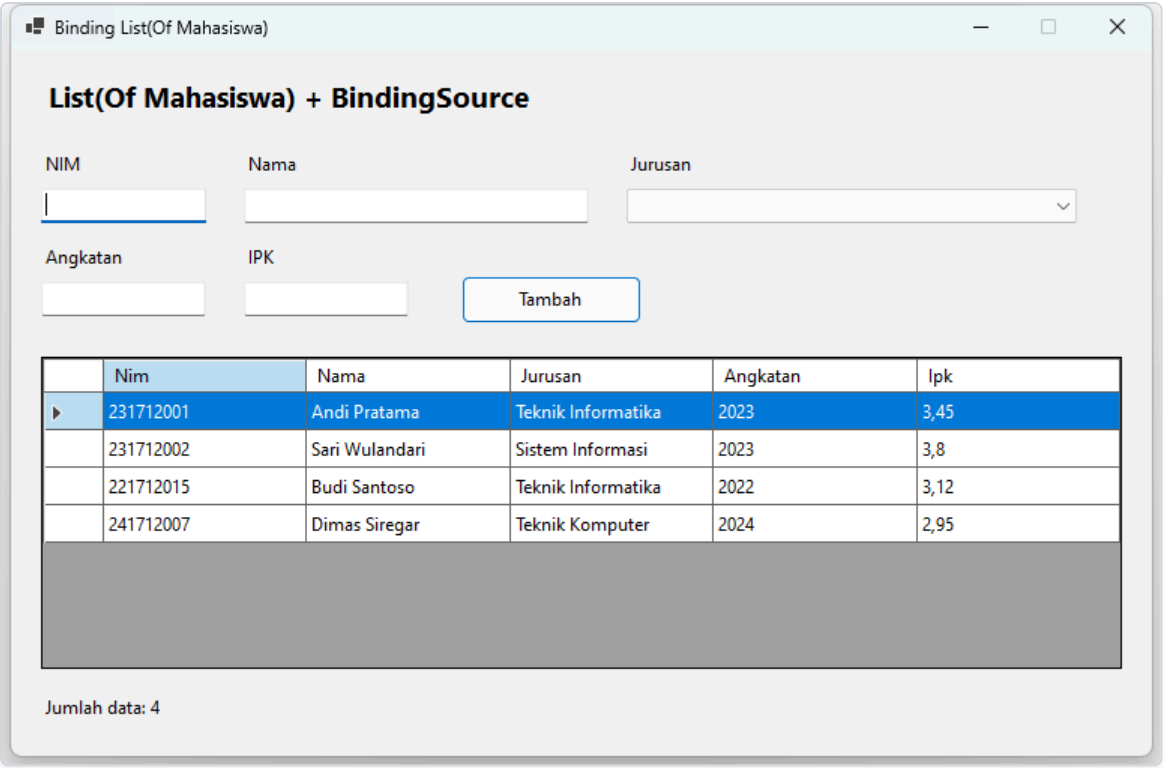
Form dilengkapi kontrol masukan di bagian atas, tabel DataGridView di bawah, serta komponen `bsMahasiswa` dan `errInput` di component tray.



Gambar 5: FrmDariList di Form Designer dengan bsMahasiswa dan errInput di component tray

2.2 Tampilan Awal Form FrmDariList

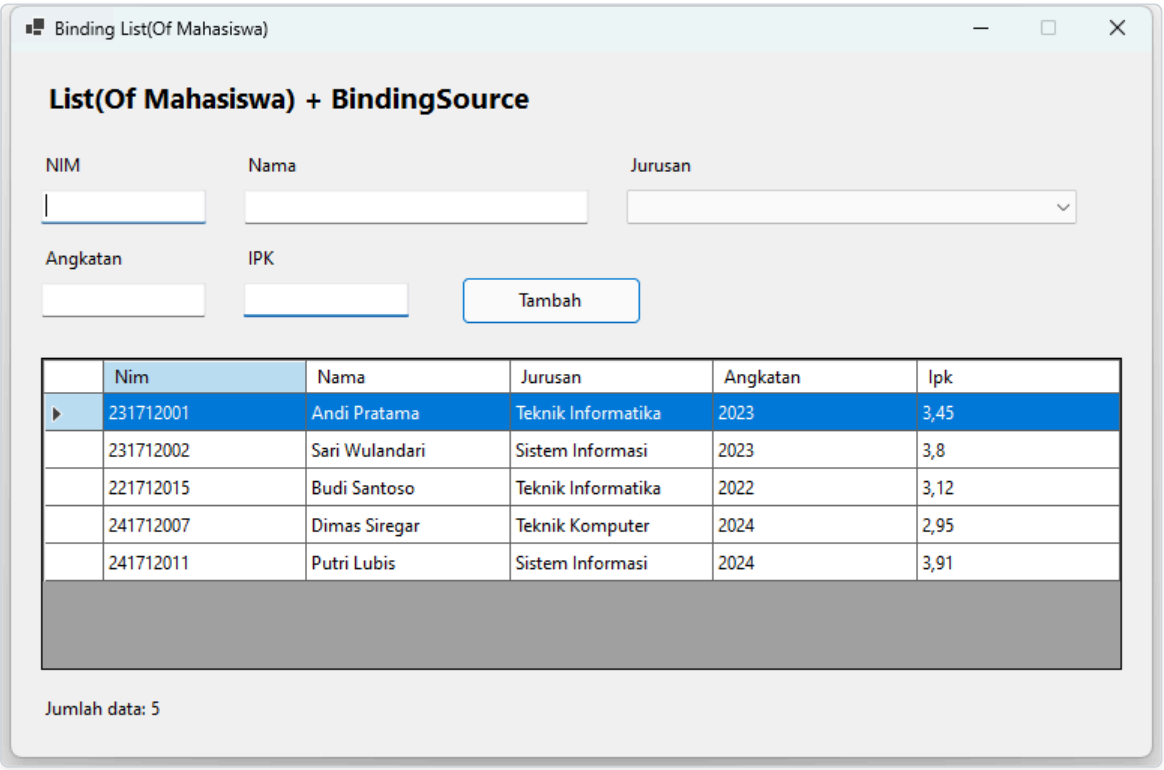
Saat form dimuat, 4 data mahasiswa contoh langsung tampil pada DataGridView melalui BindingSource.



Gambar 6: FrmDariList saat pertama kali dijalankan dengan 4 data mahasiswa

2.3 Penambahan Data Baru dengan Validasi

Menambahkan data baru (NIM: `241712011`, Nama: `Putri Lubis`, Jurusan: `Sistem Informasi`, Angkatan: `2024`, IPK: `3.91`). Karena data ditambahkan melalui `bsMahasiswa.Add(baru)`, baris kelima langsung muncul otomatis di DataGridView tanpa perlu memuat ulang data.

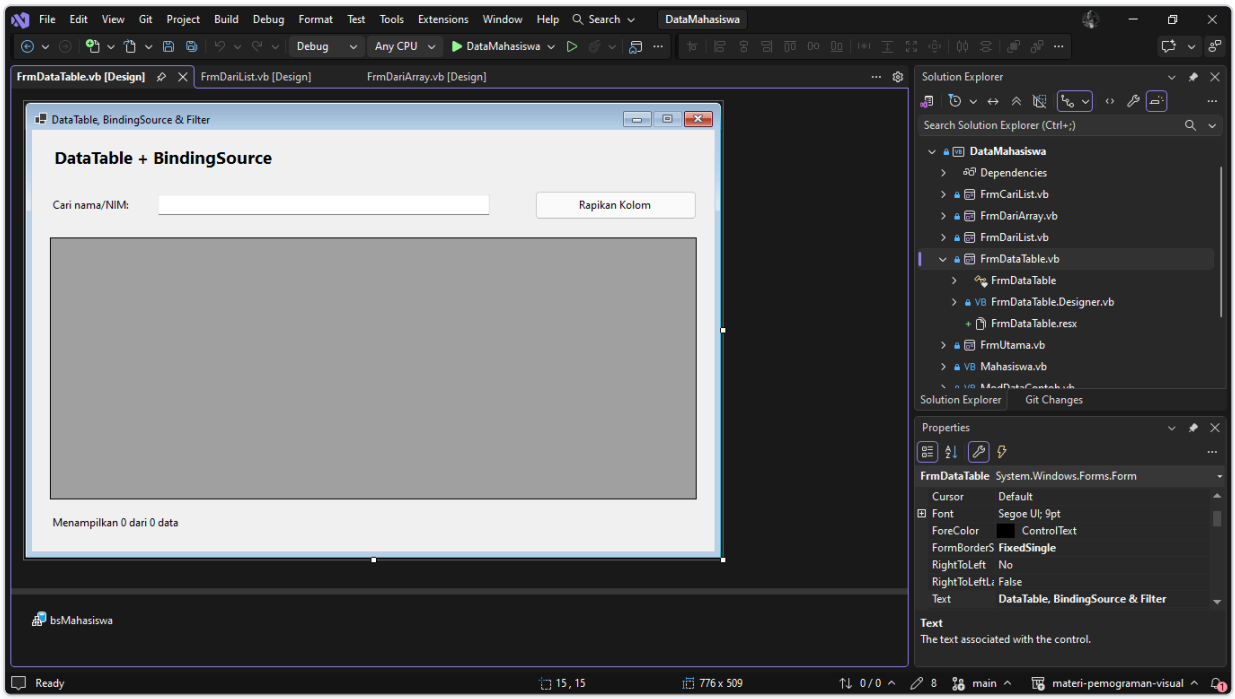


Gambar 7: FrmDariList setelah data Putri Lubis berhasil ditambahkan (jumlah data: 5)

Praktik 3: DataTable, Rapikan Kolom, dan Filter (FrmDataTable)

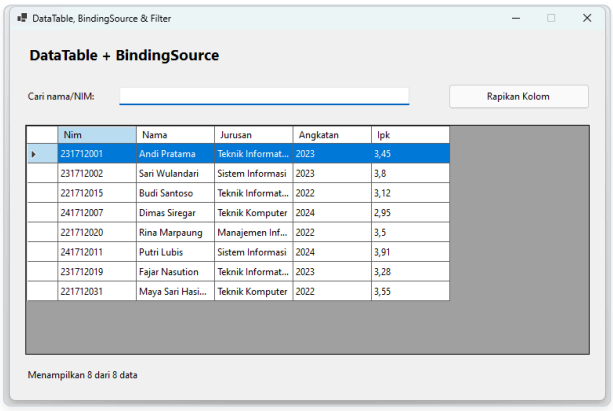
Menggunakan objek `DataTable` sebagai basis data memori. Kolom dibuat dinamis dan data disaring menggunakan sintaks ekspresi `BindingSource.Filter` dengan pengamanan karakter khusus melalui modul `ModPencarian.vb`.

3.1 Desain Form FrmDataTable

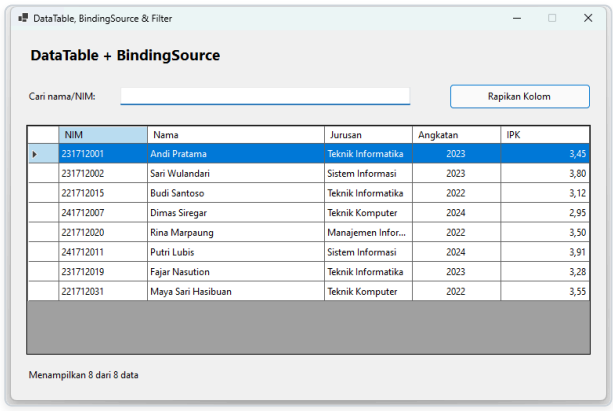


Gambar 8: FrmDataTable di Designer dengan komponen bsMahasiswa

3.2 Sebelum dan Sesudah Merapikan Kolom



Gambar 9: Sebelum tombol Rapikan Kolom diklik

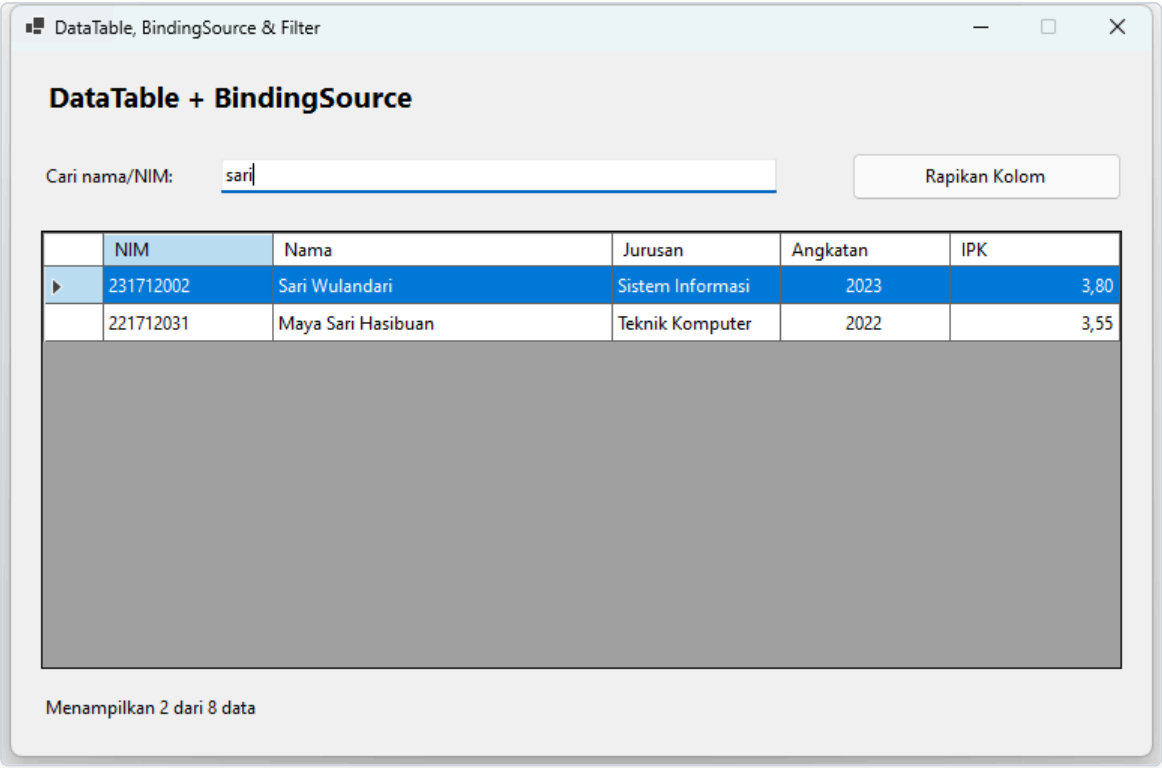


Gambar 10: Sesudah tombol Rapikan Kolom diklik

Penjelasan: Tombol **Rapikan Kolom** mengubah judul kolom menjadi huruf kapital rapi (NIM, IPK), meratakan teks IPK ke kanan dengan format 2 desimal (`N2`), meratakan Angkatan ke tengah, memperlebar kolom Nama (`FillWeight = 200`), serta mengatur `AutoSizeColumnsMode = Fill`.

3.3 Pencarian Real-Time dengan Filter BindingSource

Saat kata `sari` diketik pada kotak pencarian, ekspresi `bsMahasiswa.Filter = "Nama LIKE '%sari%' OR Nim LIKE '%sari%'"` dieksekusi secara instan dan menampilkan 2 data (Sari Wulandari dan Maya Sari Hasibuan).

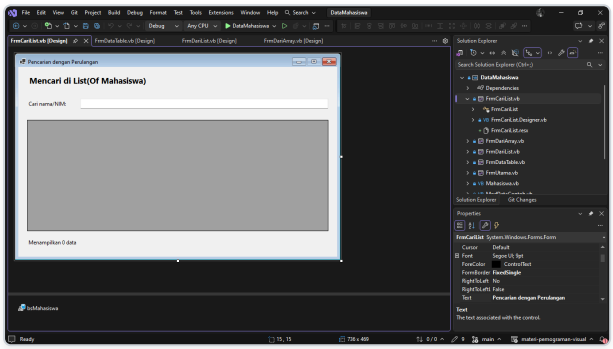


Gambar 11: FrmDataTable dengan txtCari berisi "sari", menampilkan 2 dari 8 data

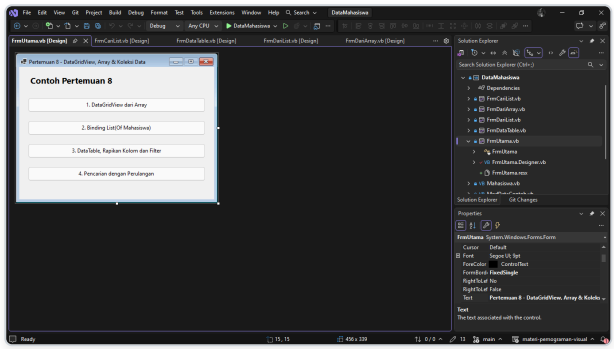
Praktik 4: Pencarian pada List(Of T) dengan Perulangan (FrmCariList)

Karena objek generic `List(Of T)` tidak memiliki properti ekspresi `Filter` seperti DataTable, proses pencarian dilakukan menggunakan perulangan `For Each` yang menyaring data secara manual dan memasukkannya ke List hasil baru.

4.1 Desain Form FrmCariList dan FrmUtama



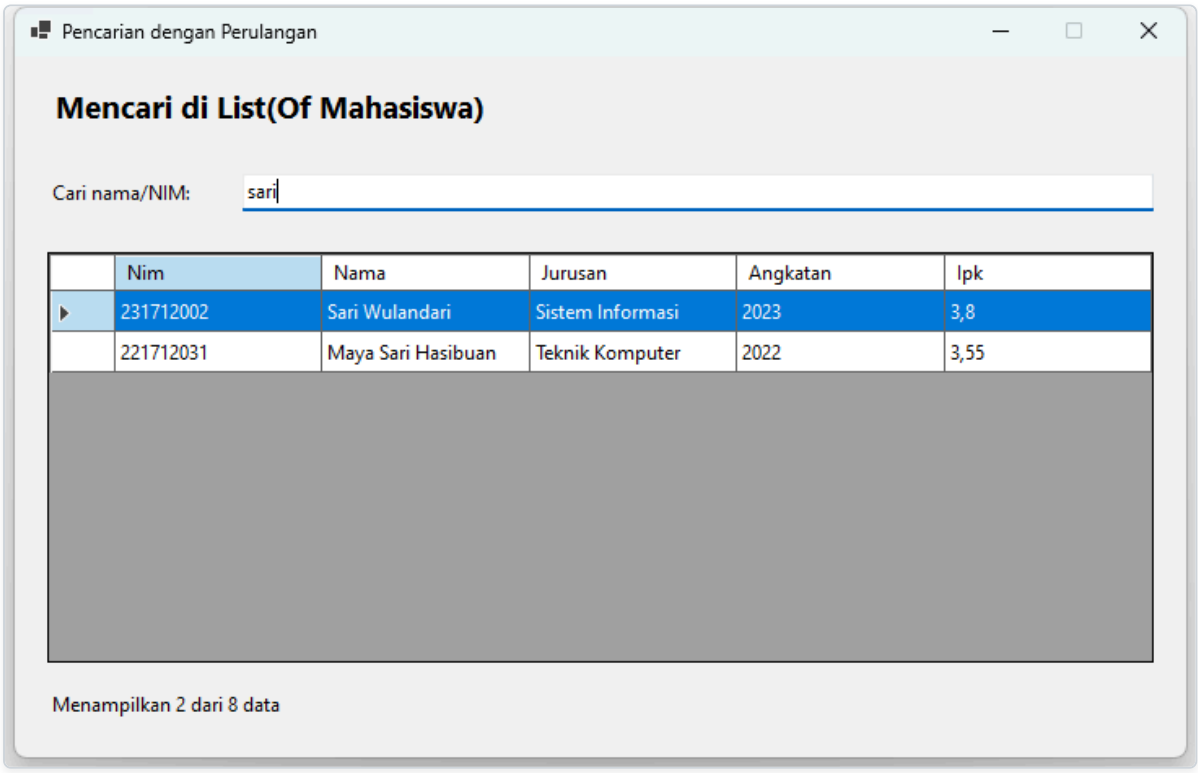
Gambar 12: FrmCariList di Form Designer



Gambar 13: FrmUtama di Designer dengan 4 tombol

4.2 Pengujian Pencarian Menggunakan Perulangan

Mengetik kata `sari` di `txtCari` memicu event `TextChanged` yang melakukan iterasi dan menghasilkan 2 baris data yang cocok.



Gambar 14: FrmCariList menyaring 2 data dengan kata kunci "sari"

Hasil Pengujian Keseluruhan Praktik 8

No	Form / Praktik	Langkah Uji	Hasil yang Diharapkan	Status
1	FrmDariArray	Klik tombol Tampilkan dan Kosongkan	Grid menampilkan 8 data; saat dikosongkan grid bersih	**Berhasil**
2	FrmDariList	Tambah data baru Putri Lubis	Baris ke-5 langsung muncul di grid melalui BindingSource	**Berhasil**
3	FrmDariList	Uji validasi isian kosong & NIM ganda	Ikon ErrorProvider muncul di samping kontrol yang salah	**Berhasil**
4	FrmDataTable	Klik Rapikan Kolom & Cari "sari"	Kolom rapi dan grid menampilkan 2 data yang cocok	**Berhasil**
5	FrmDataTable	Ketik tanda kutip (') di kotak cari	Aplikasi aman dari error filter dan tidak crash	**Berhasil**
6	FrmCariList	Cari kata "sari" atau NIM "2317"	Perulangan For Each berhasil memfilter data ke grid	**Berhasil**

Kesimpulan Praktik: Penggunaan `BindingSource` sangat memudahkan pembaruan tampilan antarmuka saat data bertambah. Untuk skenario data tabel yang dinamis dengan kebutuhan pencarian berbasis filter string, `DataTable` memberikan performa dan fleksibilitas lebih baik karena mendukung ekspresi kueri langsung di memori. Sementara itu, untuk struktur objek berorientasi objek yang ketat, `List(Of T)` sangat aman dan pencarian dilakukan menggunakan iterasi kode program.